OSSP – Skill 6

-2520030106

-D. Mohit Venkat Sai

Task 1: Process Escape Sequences

Objective: Create a program to handle escape sequences, escaped spaces, special symbols, and complex inputs.

```
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

D:\Documents\Sem 2-1\OS\2520030106_Skill> wsl
dmvs@dmvsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030106_Skill$ pwd
/mnt/d/Documents/Sem 2-1/OS/2520030106_Skill
dmvs@dmvsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030106_Skill$ nano Skill6_escape.c
```

```c
#include <stdio.h>    Explain and Fix Problem (Ctrl+Shift+.)
#include <string.h>

int main()
{
    char input[200];
    int i, j;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    for(i = 0, j = 0; input[i] != '\0'; i++)
    {
        if(input[i] == '\\' && input[i + 1] != '\0')
        {
            i++;

            if(input[i] == 'n')
                input[j++] = '\n';
            else if(input[i] == 't')
                input[j++] = '\t';
            else
                input[j++] = input[i];
        }
        else
        {
            input[j++] = input[i];
        }
    }

    input[j] = '\0';

    printf("Processed Input:\n%s", input);

    return 0;
}
```

```
dmvs@dmvsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030106_Skill$ ./Skill6_escape
Enter command: Hello\nWorld
Processed Input:
Hello
World
dmvs@dmvsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030106_Skill$
```

Task 2: Create Child Process and Execute Program

This part demonstrates:

- Creating a child using fork()
- Executing a program using exec()
- Passing arguments
- Handling execution errors
- Managing the parent process

```
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ nano Skill6_exec.c
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ gcc -Wall Skill6_exec.c -o Skill6_exec
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ ./Skill6_exec
Parent PID: 618
Child PID: 619
total 1660
-rwxrwxrwx 1 prem prem    146 Aug  2 13:30  Makefile
-rwxrwxrwx 1 prem prem  23088 Aug  8 09:55  Skill3
-rwxrwxrwx 1 prem prem   2831 Aug  9 06:45  Skill3.c
-rwxrwxrwx 1 prem prem  16096 Aug 22 09:56  Skill6_escape
-rwxrwxrwx 1 prem prem    656 Aug 22 09:56  Skill6_escape.c
-rwxrwxrwx 1 prem prem  16256 Aug 22 09:58  Skill6_exec
-rwxrwxrwx 1 prem prem    511 Aug 22 09:58  Skill6_exec.c
-rwxrwxrwx 1 prem prem 480212 Aug  8 10:10 'Week 3 Skill.docx'
-rwxrwxrwx 1 prem prem 328144 Aug  8 10:15 'Week 3 Skill.pdf'
-rwxrwxrwx 1 prem prem   2082 Aug  1 10:02  Week1Skill.c
-rwxrwxrwx 1 prem prem 147226 Aug  2 13:34  Week1Skill.docx
-rwxrwxrwx 1 prem prem 180655 Aug  2 13:34  Week1Skill.pdf
-rwxrwxrwx 1 prem prem  16720 Aug  8 09:24  Week2Skill
-rwxrwxrwx 1 prem prem   2833 Aug  8 09:23  Week2Skill.c
-rwxrwxrwx 1 prem prem 446233 Aug  8 09:36  Week2Skill.pdf
-rwxrwxrwx 1 prem prem  16168 Aug 18 06:07  sliptest_1
-rwxrwxrwx 1 prem prem    525 Aug 18 06:07  sliptest_1.c
drwxrwxrwx 1 prem prem   4096 Aug  8 09:26  test
Child execution completed
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if(pid < 0)
    {
        perror("fork");
        return 1;
    }

    if(pid == 0)
    {
        printf("Child PID: %d\n", getpid());

        execlp("ls", "ls", "-l", NULL);

        perror("exec");
        exit(1);
    }
    else
    {
        printf("Parent PID: %d\n", getpid());

        wait(NULL);

        printf("Child execution completed\n");
    }

    return 0;
}
```

```
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ nano Skill6_args.c
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ gcc -Wall Skill6_args.c -o Skill6_args
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ ./Skill6_args
Hello OSSP
Parent completed
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    char *args[] = {"echo", "Hello", "OSSP", NULL};

    pid = fork();

    if(pid == 0)
    {
        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }

    wait(NULL);

    printf("Parent completed\n");

    return 0;
}
```